Task 4: Password Cracking Lab

This report documents the password cracking exercises performed as part of the ethical hacking lab. Using access gained in Task 3 (via the vsftpd 2.3.4 backdoor exploit), the /etc/shadow file was extracted from the Metasploitable target and password hashes were cracked using John the Ripper.

Tools used: Metasploit Framework (for initial access), John the Ripper 1.9.0-Jumbo (for hash cracking), nano (to save hashes to a file). Platform: Kali Linux.

4.1 Background — How Password Hashing Works

Linux systems store passwords in /etc/shadow rather than /etc/passwd. Each entry contains a username and a hashed version of the password in the format idsalt$hash, where the id indicates the hashing algorithm used. In this lab, the hashes use 1 indicating the MD5crypt algorithm.

Password cracking involves running a dictionary of candidate passwords through the same hashing algorithm and comparing the output against the stored hash. A match reveals the original plaintext password.

4.2 Extracting Password Hashes from Metasploitable

With root shell access obtained in Task 3, the /etc/shadow file was read directly. This file is normally only readable by root, making initial exploitation a prerequisite for this step.

Command run inside Metasploit shell

```
hostname
cat /etc/shadow
```

Figure 1: cat /etc/shadow output — password hashes visible for all system accounts

The shadow file revealed hashed passwords for all system accounts. Accounts with * or ! in the hash field have no valid password (login disabled). Accounts with actual hash strings are targets for cracking. The notable entries with real hashes were:

- root:1/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747 — root account
- sys:1fUX6BPOt$Miyc3Up0zQJqz4s5wFD9l0:14742 — sys account
- klog:1f2ZVMS4K$S R9XkI.CmLdHhdUE3X9jqP0:14742 — klog account
- msfadmin:1XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684 — msfadmin (main user account)

These four hashes were selected for cracking as they represent real user accounts that could provide SSH login access after cracking.

4.3 Saving Hashes to a File on Kali

A new terminal was opened on the Kali machine. The nano text editor was used to create hashes.txt and paste in the four hash entries copied from the Metasploitable shell session.

Command

```
nano hashes.txt
```

Figure 2: nano hashes.txt opened in a new terminal on Kali to save the extracted hashes

The file was saved with Ctrl+X → Y → Enter. This creates a local file on Kali containing only the lines with real password hashes, formatted exactly as they appeared in /etc/shadow.

4.4 Installing John the Ripper & Cracking the Passwords

John the Ripper was confirmed to be installed. Running the installation command verified the version and confirmed no reinstallation was needed.

Command

```
sudo apt install john
john hashes.txt
```

Figure 3: apt install confirms john 1.9.0-Jumbo-1 is already installed — cracking begins immediatel

- John automatically detected the hash format as md5crypt (1) and warned it could also be md5crypt-long. It loaded 4 hashes with 4 different salts and began cracking using 2 OpenMP threads.
- Cracking sequence observed:
- Phase 1 — Single rules: John tries variations of the username itself as the password. Result: msfadmin:msfadmin cracked immediately; the password is the same as the username.
- Phase 2 — Wordlist (password.lst): John runs through /usr/share/john/password.lst, a list of common passwords. Results: 123456789 cracked for klog, batman cracked for sys.
- Phase 3 — Incremental ASCII: John begins brute-forcing all character combinations. This phase is ongoing for the root hash.

4.5 Viewing Cracked Passwords

The --show flag displays all passwords cracked so far from the specified hash file.

Command

```
john --show hashes.txt
```

Figure 4: john --show hashes.txt — 3 passwords cracked, 1 remaining

Results:

- sys : batman
- klog : 123456789
- msfadmin : msfadmin

3 password hashes cracked, 1 left (the root hash was not cracked during the session — it uses a stronger or more unique password).

These credentials can now be used to log in via SSH, Telnet, or any other service discovered during the Nmap scan.

4.6 Using Cracked Credentials — SSH Login

The cracked credentials were tested against the Metasploitable SSH service to verify they work for real authentication.

Commands

```
ssh klog@192.168.56.101
# Error: no matching host key type
ssh -o HostKeyAlgorithms=+ssh-rsa klog@192.168.56.101
```

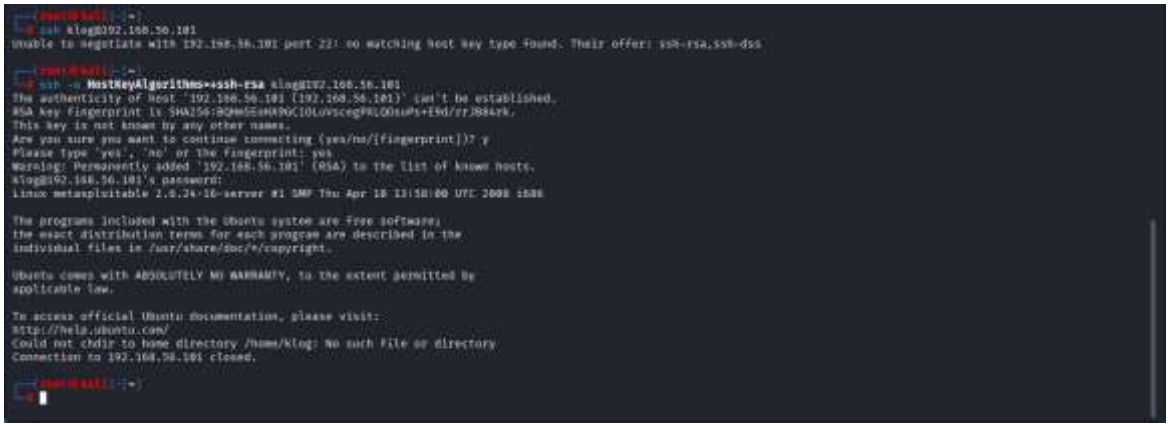

Figure 5: SSH login with klog:123456789 — successful after host key algorithm override

Challenge encountered: The first SSH attempt failed with "no matching host key type found. Their offer: ssh-rsa, ssh-dss". Modern OpenSSH versions on Kali have disabled legacy RSA-SHA1 key types by default because they are considered cryptographically weak.

Resolution: The -o HostKeyAlgorithms=+ssh-rsa flag was added to explicitly allow the legacy key type. The connection was then established successfully. The RSA fingerprint was accepted, the password 123456789 was entered, and the Ubuntu 8.04 welcome banner appeared.

The session then closed immediately with "Could not chdir to home directory /home/klog: No such file or directory". This means the klog account has no home directory — it is a system/service account. The login still confirmed that the cracked password is valid.

4.7 Observations and Lessons Learned

- Weak passwords are the norm on default systems. msfadmin:msfadmin (same as username) and klog:123456789 (sequential digits) were cracked in seconds. These are the most common password patterns found in real-world breaches.
- John the Ripper is highly efficient. It cracked 3 of 4 hashes using only the built-in wordlist and single rules, without needing rockyou.txt or extended brute force.
- MD5crypt (1) is weak. This algorithm dates from the 1990s. Modern Linux systems should use SHA-512 (6) or yescrypt, which are dramatically more resistant to cracking.
- Root hash was not cracked. The root account uses a stronger or more unique password that resisted wordlist and single-rule attacks. This highlights that even on a deliberately vulnerable machine, not all passwords are trivially weak.
- SSH legacy compatibility issues are real. Older servers running OpenSSH 4.7 require explicit algorithm overrides in modern clients. This is increasingly common when testing legacy infrastructure.
- Credential validation completes the chain. The full attack chain — scan → exploit → hash extraction → crack → SSH login — demonstrates how a single unpatched FTP vulnerability leads to complete system compromise with valid credentials for multiple user accounts.

4.8 Security Implications

- The password cracking exercise highlights several critical real-world security issues:
- Default and trivial credentials are catastrophic. Any user whose password equals their username or uses a common pattern is an immediate compromise waiting to happen. Password policies must enforce complexity and length.
- Hash algorithm matters. MD5crypt hashes are crackable in seconds with modern GPUs. All Linux systems should be configured to use SHA-512 or yescrypt.
- Shadow file exposure requires root. The fact that root access was needed to read /etc/shadow is an important control; it means attackers must already have elevated access before reaching this stage. Protecting against initial exploitation (patching vsftpd) breaks the entire chain.
- Cracked passwords enable lateral movement. Once klog:123456789 and msfadmin:msfadmin are known, an attacker can authenticate to SSH, Telnet, MySQL, and any other service accepting those credentials without triggering further exploits.
- Never run Metasploitable on a production network. These deliberately weak credentials and services would compromise any real network in minutes.